HACKATHON LA MÊLÉE 2026 · DÉFI ENEDIS

Crisis dispatch: team brief

Problem, data, objective function, constraints, algorithm and roadmap

*Version 1 · 9 October 2026 · Working document: items marked **TO DECIDE** need a team decision before coding.*

The brief in 5 lines

1. We build a **decision-support demonstrator**: it replays a crisis, recommends **which team goes to which incident and in what order**, and explains why. A human always makes the final decision.
2. We minimise one score: **Σ (weighted customers × hours without power)**, summed over incidents.
3. The core difficulty: in big crises there is **more work than the 63 teams can absorb in one 12 h shift**. Choosing the **order** of repairs matters far more than travel distance.
4. The approach: a **simulator** plus a **priority-based greedy plan improved by local search**, re-planned at every event and compared against simple baselines on **all 100 crises**.
5. **MVP first** (replay, plan, score, map, benchmark), **then differentiators** (cost of an override, hindsight bound, staggered shifts, pre-positioning, LLM copilot).

#	Section	#	Section
1	The challenge in plain words	7	Code architecture and interfaces
2	What the data tells us	8	MVP vs differentiators
3	The model: objects and their life cycle	9	Roadmap and division of work
4	Objective function	10	Decisions to settle
5	Constraints	11	Limits and risks
6	Algorithm	12	Glossary FR / EN

1. The challenge in plain words

During a climate crisis (storm, lightning, strong wind), many outages happen at once. Enedis has a fixed number of field teams, already positioned at operational sites. Today, the choice of which team goes to which outage is made by hand. We build a tool that **recommends**, as the crisis unfolds, which team should go where and in what order, so that power comes back **as fast as possible for as many customers as possible**, with priority for sensitive customers. The crisis manager stays in charge: they accept, modify or reject every recommendation.

It IS	It is NOT
A logistics / scheduling problem at the moment of the crisis	A weather or outage forecasting problem
A simulation of the crisis as incidents arrive over time	A tool that already knows the future
A recommendation system with explanations	An autopilot that sends teams on its own
Measured against simple reference strategies	Judged on visuals alone

The three modes described in the README

Mode	What it means	Our plan
Jour J (crisis day)	Incidents arrive; the tool assigns teams and recomputes as things change	**MVP core**
Rejeu (replay)	Replay a historical crisis and compare several strategies	**MVP**: it is how we prove performance
Préparation	Before the storm: suggest where to pre-position teams	Differentiator (section 6.10)

How we will be judged (README section 9)

Criterion	What the jury looks at	What we will show
Business relevance	Answers the challenge, respects the human decision	Accept / modify / reject on every recommendation
Performance	Beats a reference strategy on the N×T indicator	One chart: our score vs baselines on 100 crises
Robustness	Handles shocks: 50% of teams gone after 48 h, 2 successive crises, saturation	Scenario switches in the simulator
Explainability	Recommendations are understandable and contestable	Decisive factors + cost of the alternative
Technical quality	Structured, reproducible, configurable, documented	Git repo, README, one config file
User experience	The visualisation helps understand and steer	Map + timeline + recommendation panel, in French

2. What the data tells us

We received four CSV files (semicolon-separated, UTF-8) plus the README and the pitch deck. All numbers below are computed from the files themselves.

File	Rows	Key columns	Meaning
poste_v2.csv	33 288	id_poste, commune, lat/lon, nb_clients, poste_avec_client_sensible	One substation. Customers 5 to 150 (generated). Sensitive is a 0/1 flag.
coupures_v2_100_crises.csv	15 681	numero_crise, id_incident, id_poste, causeInterruption, dateHeureIncident, dureeDepannage	One incident = one substation down, at a given time, with a repair duration of 2, 4 or 8 h.
agents_v2.csv	18	id_site, commune, departement, lat/lon, nb_equipes	18 operational sites holding 63 teams in total.
insee_v2.csv	1365	code_insee, commune, lat/lon, nb_postes	Commune reference (for display / aggregation).

Links: `coupures.id_poste → poste.id_poste`, `poste.code_insee → insee.code_insee`*; distances are computed from site coordinates to substation coordinates.*

Key numbers and what they imply

Finding	Consequence for our design
100 crises (2016-2018), 50-250 incidents each (median 166), arriving over 4-8 h. Crises never overlap.	We have a full test bench: every strategy can be scored on 100 crises, not one demo. Two successive crises can be simulated by chaining two.
Work per crisis: median 11.6 h of repair per team, max 18.5 h. 48 crises exceed 12 h per team.	Teams get saturated and a queue forms. The 12 h / 11 h work rule really bites on big crises.
Repairs last 2, 4 or 8 h. Median distance to the nearest site is 11 km (90% under 22 km), i.e. about 15-30 min by road.	Repair time dominates travel. The decision is mostly about **order** and about balancing load between sites. Straight-line distance × detour factor is enough.
Repair duration does not depend on the cause (every cause appears with 2, 4 and 8 h).	Cause is a label only; we cannot predict duration from it. Duration uncertainty can be modelled as noise (bonus).
Customers per substation are uniform 5-150. 1 664 substations (5%) are flagged sensitive.	Weight = customers × (1 + β × sensitive). β is a visible setting.
Gers holds about a third of incidents but under a quarter of teams (see chart below).	Pre-positioning teams before the storm has measurable value (differentiator).

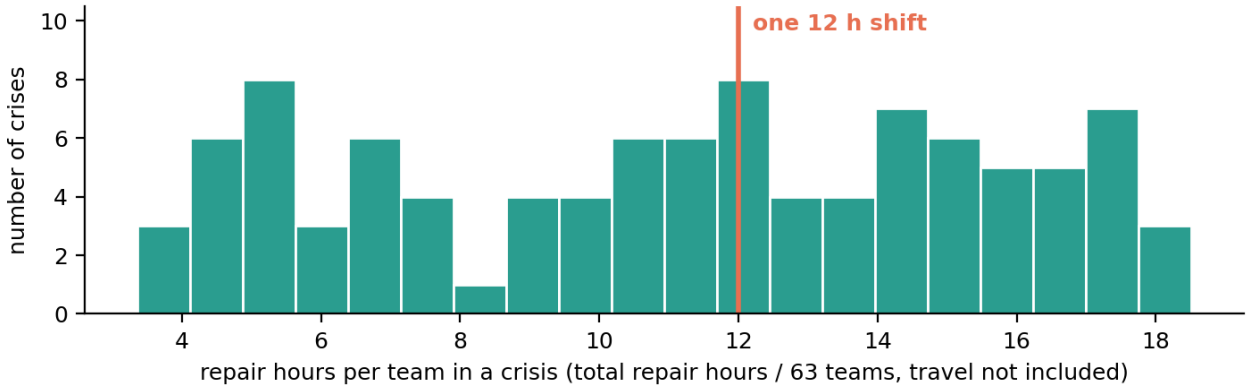


Figure 1. Load per team in each of the 100 crises. Right of the orange line, the teams cannot finish the work in one 12 h shift even if it were perfectly balanced and travel were free (48 crises).

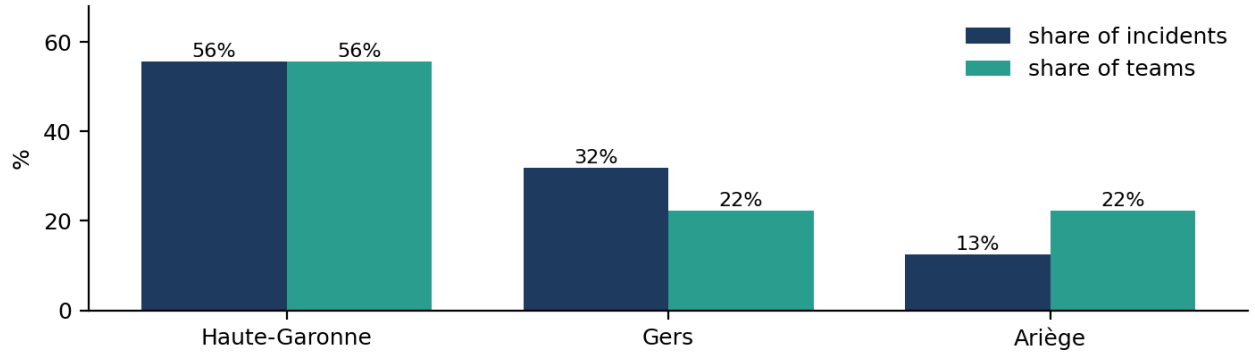


Figure 2. Share of incidents vs share of teams per department. Gers is under-resourced relative to the incidents it gets; Ariège has more teams than its share of incidents.

Data quality points to handle (and mention as limits)

- 5 incidents reference a substation that does not exist in poste.csv: drop them and log it.
- 43 cases of the same substation failing twice within one crisis: treat as separate incidents (or merge: TO DECIDE).
- The README says 20 sites; the file has 18. We use the file.
- Customer counts and incident locations are synthetic (stated by the README); no personal data is involved.

3. The model: objects and their life cycle

The README recommends an object-oriented model. These are the objects we will code, with the attributes we use.

Object	Source	Attributes we use
Poste (substation)	poste.csv	id, position (lat/lon), customers, sensitive flag, commune, department
Incident	coupures.csv	id, substation, arrival time, cause, repair duration, status, restoration time
Site	agents.csv	id, position, number of teams based there
Équipe (team)	derived from agents.csv	id, home site, current position, status, current job, work time used in this shift, next rest
Scénario	assembled	one crisis = substations + its incidents + teams + configuration
Plan / Décision	computed	for each team, an ordered list of incidents; each decision is assign, reassign, hold in reserve or reposition

Life cycle

Incident	Arrived → Waiting → Team en route → Being repaired → **Restored**
Team	Available → Travelling → Repairing → (Available or Idle) → Resting 11 h after 12 h of work → Available …

Notation used in the rest of the document

Symbol	Meaning	Where it comes from
i	an incident	coupures.csv
k	a team	agents.csv (nb_equipes per site)
r_i	arrival time of incident i	dateHeureIncident
p_i	repair duration of incident i (2, 4 or 8 h)	dureeDepannage
n_i	customers cut off by incident i	nb_clients of its substation
s_i	1 if the substation has sensitive customers, else 0	poste_avec_client_sensible
β	extra weight for sensitive customers (setting, default 3)	our choice

Symbol	Meaning	Where it comes from
w_i	weight of incident i = $n_i \times (1 + \beta \times s_i)$	computed
$\tau(a, b)$	travel time from point a to point b	distance × 1.3 / average speed
C_i	time at which repair of i ends = customers restored	computed by the plan
$C_i - r_i$	total outage time of i = waiting + travel + repair	computed

4. Objective function

4.1 The main objective, in words

From the moment an incident appears until its repair ends, its customers are in the dark. Each incident therefore accumulates **customer-hours without power**. We want the **total** over all incidents to be as small as possible, giving extra weight to sensitive customers.

```
minimise   SCORE = Σ over incidents i of   w_i × (C_i − r_i)

           w_i       = n_i × (1 + β × s_i)          customers, boosted if sensitive
           C_i − r_i = waiting + travel + repair    hours without power
```

> **Note on the README formula**
> The README writes the formula as *maximiser Σ Nbclients × Temps de coupure*. Taken literally, that would reward long outages. The intent is clearly to **minimise** it, and the README itself says the objective is "to be adjusted and justified". We state this reading explicitly in our README.

Example of the weight: with β = 3, a sensitive substation with 50 customers weighs 50 × (1 + 3) = 200, the same as a normal substation with 200 customers.

4.2 Worked example: why order matters

One team, three incidents known at time 0, travel ignored. A: 150 customers, 8 h repair. B: 60 customers, 2 h. C: 100 customers, 4 h. The score is the sum of customers × time at which each one is restored.

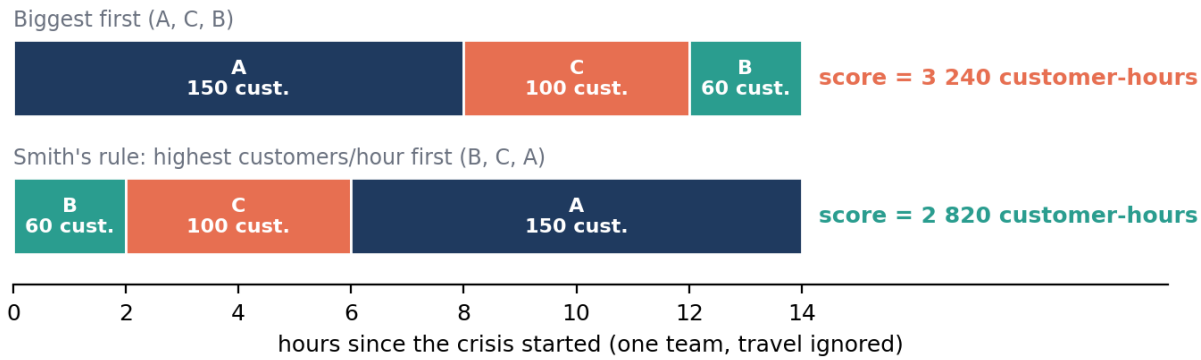


Figure 3. Repairing the biggest incident first looks natural but scores worse. Serving the best customers-per-hour ratio first (B: 30/h, C: 25/h, A: 18.75/h) restores 13% fewer customer-hours of outage.

> **Smith's rule: the core intuition**
> For a single team with no travel, serving incidents in decreasing order of $\mathbf{w_i}$ / $\mathbf{p_i}$ (customers restored per hour of work) is **provably optimal**. This is known as Smith's rule (1956). Our priority index (section 6.2) extends it with travel time. It also gives a simple explanation for the manager: *"this incident goes first because it brings back the most customers per hour of work"*.

4.3 The full objective (configurable)

The README invites us to enrich the objective with business penalties. We use a weighted sum where every weight λ is visible in the configuration and in the interface:

```
minimise   Σ w_i × (C_i − r_i)                    customer impact (main term)
         + λ_stab × number of teams re-routed while en route      plan stability
         + λ_trav × total travel time                             safety / cost, tie-breaker
         + λ_eq   × max over i of (C_i − r_i)                     fairness: no forgotten incident
```

Term	Why it is there	Default
Stability	A manager will not trust a plan that reshuffles everything at every update. Re-routing a team already on the road must be worth it.	on, moderate
Travel	Less driving in storm conditions is safer. Mostly breaks ties between equal options.	small
Fairness	Without it, a small remote substation (e.g. mountain Ariège) could wait forever behind bigger ones.	small

4.4 Indicators we report (but do not optimise directly)

- Weighted customer-hours (the score) and unweighted customer-hours (the README's N×T)
- Restoration time of sensitive customers
- % of customers restored after 4 h, 8 h, 12 h, 24 h
- Saturation: number of incidents waiting with no team available, over time
- Team utilisation and maximum outage per department

5. Constraints

Hard constraints must always hold (safety and work rules come first, as the README insists). Every rule is a parameter in the configuration, never a hard-coded assumption, because the organisers may change it.

#	Constraint	In plain words	Setting (default)
C1	Every incident is served	Each incident gets exactly q_i teams. Today $q_i = 1$. If $q_i > 1$, repair starts when the last team arrives.	teams_per_incident = 1
C2	One job at a time, in sequence	A team does one incident, then travels to the next. Next start ≥ previous end + travel time. The first trip starts from the home site.	fixed
C3	No anticipation	A team cannot leave for an incident before the incident is known. Replays must not peek at the future.	fixed
C4	No interruption	Once a repair starts, it runs until it is finished.	fixed
C5	Work window	A team works at most 12 h in a row, then rests 11 h. Travel + repair must fit **entirely** in the window: never start a job you cannot finish.	shift_h = 12, rest_h = 11, travel_counts = True, allow_split = False
C6	Where the team rests	The team rests where it is and restarts from there (or goes back to base).	rest_at = 'site' (TO DECIDE)
C7	Locked decisions	Teams already travelling or repairing keep their job. An operator choice forces (or forbids) an assignment.	from the interface
C8	Unavailability shocks	A team becomes unavailable from some time onward (README stress test: 50% of teams after 48 h).	unavailability = []
C9	Sensitive deadline (optional)	Sensitive incidents must start within D hours.	off

The trap hidden in C5
If all 63 teams start their shift at the start of the crisis, **they all go to rest at the same time**, at hour 12, and nobody is in the field for 11 hours. That is exactly when big crises still have a queue (Figure 1). The fix is a **mobilisation time per team**: hold some teams in reserve and start them later, so that the work is covered around the clock. The README calls this *maintien en réserve*. The MVP starts everyone at hour 0; staggered mobilisation is one of our levers (section 6.9).

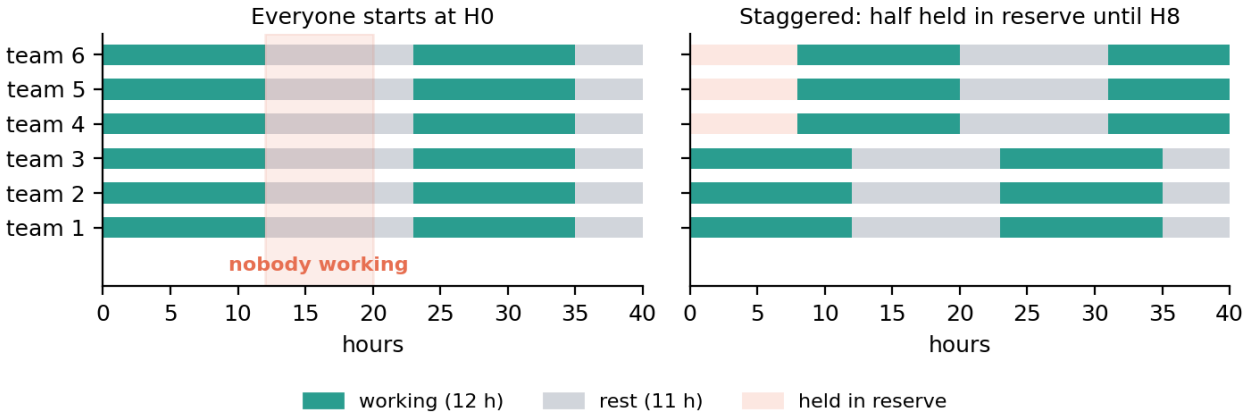


Figure 4. Schematic with 6 teams. Left: everyone starts together and the field is empty between hours 12 and 23. Right: staggering keeps teams working through the night.

Questions to settle on the constraints (TO DECIDE)

- Does travel count as work time? Our default: yes.
- Can a repair cross the end of a shift? Our default: no (safety).
- Does a team rest on site or back at base? Our default: on site.

- Can idle teams reposition towards busy areas while waiting? The README's step T5 mentions *repositionner*. Our default: yes, as a recommendation.
- Do all teams start at hour 0, or do we allow staggered mobilisation? MVP: hour 0; lever later.

6. Algorithm

6.1 Why not an exact solver

In scheduling terms, the problem is *parallel machines with release dates, sequence-dependent set-up times and weighted completion time*, written $R \mid r_j, s_{ij} \mid \Sigma\, w_j C_j$, plus work windows. Even the single-team version with release dates is NP-hard. With about 165 incidents and 63 teams, re-solved every few seconds, an exact solver is out of reach. We use a **heuristic**, and an exact solver only offline on small cases to measure how good the heuristic is.

6.2 The priority index

```
priority(i, k) = w_i / ( τ(position of team k, i) + p_i )

   = weighted customers brought back per hour that team k spends on incident i
```

This extends Smith's rule (section 4.2) to travel. High priority means many customers, a short repair and a team close by.

6.3 Reference strategies (baselines), required by the README

Baseline	Rule
B0-a First come, first served	The oldest waiting incident gets the nearest available team.
B0-b Nearest	Each team that becomes free takes the nearest waiting incident.
B0-c Business rule	The incident with the most customers (sensitive first) gets the nearest available team.

6.4 Our online policy (MVP): plan everything, commit only the next step

The README describes a T1-T5 decision loop. We implement it as an **event-driven** loop. It recomputes the plan whenever something happens: a new incident, a team becomes free, a shift ends, or the operator overrides.

```
on every event at time t:
  T1  update the list of unresolved incidents (new arrivals included)
  T2  snapshot the teams: position, status, work time left in the shift
      lock everything already committed (C7)
  T3  GREEDY: repeatedly insert the (team, incident) pair with the best priority
      that still fits in the team's shift (C5), until all incidents are placed
      IMPROVE: local search for N milliseconds on the full objective
  T4  dispatch only the NEXT job of each idle team  -> shown as recommendations
  T5  when a repair ends: customers restored, team freed, loop again
```

Because we only commit the next step, the rest of the plan stays flexible and re-planning is natural. The stability term (λ_{stab}) prevents useless reshuffles.

6.5 Local search moves

Move	What it does
Relocate	Move one incident from team A's list to team B's list (any position).
Swap	Exchange two incidents between two teams.
Reorder	Change the position of an incident inside one team's list.

A move is kept only if it lowers the full objective and respects every hard constraint. The search stops after a fixed time budget (for example 200 ms), so the tool stays real-time.

6.6 Hindsight bound (differentiator)

Offline, we solve each crisis **knowing all incidents in advance**, with a longer time budget (exact CP-SAT on small crises). No real policy can beat this, so it is a lower bound on the score. That lets us say: *"our policy closes X% of the gap between the baseline and the best achievable"*. Few teams will show this.

6.7 Counterfactuals: the cost of an override and contrastive explanations

To answer "what if team 7 goes to Muret instead?", we force that assignment (C7), re-solve the rest, and compare the two scores. The same mechanism powers three features:

- **Override cost:** when the operator changes a recommendation, show "this choice costs +X customer-hours".
- **Contrastive explanation:** "team 3 goes to A rather than B; the alternative would cost +Y".
- **LLM copilot tools:** the LLM calls this function; it never computes numbers itself.

6.8 Explanation of each recommendation

Every recommendation shows the factors behind it: customers affected, sensitive or not, travel time, repair time, work time left for the team, its priority rank, and the cost of the best alternative. A confidence or limits line is included where relevant, as the README asks.

6.9 Staggered mobilisation (differentiator)

Choose a start time for each team (all at hour 0, or some held in reserve) so that the field is never empty during rest periods. We evaluate by simulation: try a few patterns (0 %, 25 %, 50 % held back for 4, 6 or 8 h) and keep the best average score over the 100 crises.

6.10 Pre-positioning (differentiator, README preparation mode)

Before the storm, move teams between sites (for example from Toulouse towards Gers). We search over moves of one team from one site to another and evaluate each layout by simulating our policy on historical crises. To be honest, we evaluate on crises **not used** to choose the layout (train / test split).

6.11 Travel-time model

τ = straight-line distance × 1.3 (road detour) ÷ average speed (default 50 km/h, slower for mountain sites such as Ariège and Luchon). This is justified because the median trip is about 11 km while repairs take 2-8 h. Real road routing (OSRM) is a possible later upgrade.

7. Code architecture and interfaces

Layer	Responsibility	Rule
DATA	Load and clean CSVs, build Poste / Incident / Team / Site / Scenario, compute travel times	The only code that knows the CSV format
SIMULATOR	Event clock; applies constraints C2-C8; computes the score and KPIs	The single source of truth for numbers
ENGINE	Policies (baselines, ours), counterfactuals, hindsight bound	The only source of decisions
INTERFACE	Map, timeline, recommendation panel (French), LLM copilot	Never computes; only displays and asks the engine

The three contracts that let us work in parallel

```
load_scenario(crisis_id, config)        -> Scenario
policy.decide(state, locks)             -> Plan            # ordered jobs per team
simulate(scenario, policy, config)      -> Result          # timeline of events
score(result, weights)                  -> Score + KPIs
counterfactual(state, forced, policy)   -> Plan + delta score
```

Proposed repository layout

```
enedis-hackathon/
  data/raw/            original files (never edited)
  config/default.yaml  β, λ weights, shift rules, speeds, time budget
  src/data/            loaders, cleaning, travel times
  src/sim/             simulator, events, score, KPIs
  src/engine/          baselines, greedy, local search, counterfactual, bound
  src/app/             Streamlit app (map, timeline, recommendations)
  src/copilot/         LLM layer (tools that call the engine)
  bench/               100-crisis benchmark, robustness scenarios
  docs/                this brief, hypotheses and limits
```

Technology

- Python everywhere (recommended by the README).
- Streamlit for the interface: fastest path to a working demo in 48 h (suggested by the README). Map with pydeck or folium.
- OR-Tools CP-SAT only for the offline hindsight bound on small crises.
- An LLM API for the copilot, calling engine functions as tools.

8. MVP vs differentiators

MVP: must work end-to-end before anything else

#	Item	Done when…	Criterion served
M1	Data loaders + domain objects	One crisis loads into a Scenario; orphans logged	Technical quality
M2	Simulator + score + KPIs	A replay runs with constraints C1-C8 and prints the score	Performance
M3	3 baselines + our greedy policy	Both run on the same crisis through the same simulator	Performance
M4	Benchmark on 100 crises	One chart: our score vs baselines, average and spread	Performance, robustness
M5	Interface v1 (French)	Map of the 3 departments, time slider, recommendation list with accept / reject	UX, business relevance
M6	Explanation factors	Each recommendation shows its factors and its rank	Explainability
M7	README + hypotheses + limits	Someone outside the team can run it	Technical quality

Differentiators, in order of value for effort

#	Feature	Why it stands out	Depends on
D1	Override cost + contrastive explanations	Makes the tool contestable, which is exactly the explainability criterion	M3
D2	Local search on top of greedy	Better scores, still real-time	M3
D3	Hindsight bound	"We close X% of the gap to the best possible": a strong, rare argument	M4
D4	Robustness scenarios	50% of teams lost, 2 successive crises, few teams: one switch each	M2
D5	Staggered mobilisation	Solves the everyone-rests-at-once trap; a real operational insight	M2
D6	Pre-positioning mode	Covers the README's preparation mode with a data-backed result	M4
D7	Levers analysis	"One more team at Auch saves X customer-hours": what a manager acts on	M4
D8	LLM copilot (French)	Questions and what-if in natural language, numbers always from the engine	D1

9. Roadmap and division of work

Hours are counted from the moment we start coding (H0). Adjust to the real hackathon schedule; the order matters more than the exact times.

Phase	When	Goal	Milestone (definition of done)
0. Align	H0 - H1	Read this brief, settle the TO DECIDE items, create the repo skeleton, assign owners	Repo exists, config file agreed, owners named
1. MVP core	H1 - H8	M1, M2, M3	One crisis replayed end-to-end; our score vs one baseline printed
2. MVP complete	H8 - H16	M4, M5, M6 + D2	100-crisis chart; Streamlit replay with recommendations and factors
3. Differentiators	H16 - H30	D1, D3, D4, D5, then D6 / D7 / D8 as time allows	Each feature visible in the demo, or dropped cleanly
4. Freeze and pitch	last ~6 h	M7, pitch deck, demo rehearsal, backup video	Feature freeze; demo runs twice without errors

Workstreams

Stream	Contents	Can start	Owner
A. Data	Loaders, cleaning, travel times, Scenario objects	Now	
B. Simulator + score	Event clock, constraints, KPIs	Now (on a mock Scenario)	
C. Engine	Baselines, greedy, local search, counterfactual	Now (on the State / Plan contract)	
D. Benchmark	100 crises, hindsight bound, robustness scenarios	After B + C	
E. Interface	Streamlit map, timeline, recommendation panel	Now (on a mock Result)	
F. Explanations / LLM	Factor display, override cost, copilot	After C (counterfactual)	

With 4 people, one possible split: (1) A + B, (2) C, (3) E, (4) D + F. Agree on the contracts in section 7 first, so nobody waits on anybody.

Demo script (target: 5 minutes)

- The map at crisis start: sites, teams, first incidents appearing.
- Recommendations stream in with their factors; the manager accepts them.
- The manager overrides one: the tool re-plans and shows what the override costs.
- The benchmark chart: our policy vs baselines on 100 crises (and the gap to the hindsight bound).
- Stress test: half the teams disappear; the plan adapts.
- Copilot question in French, answered with figures taken from the engine.

10. Decisions to settle (with our proposed default)

Decision	Proposed default	Why
Reading of the README formula	Minimise Σ w(C − r)	Maximising outage time makes no sense
β (sensitive weight)	3, with a slider	Visible and adjustable; the organisers gave no value
Travel counts as work	Yes	Safety and work-time rules
Job crossing the end of a shift	Forbidden	Safety first
Rest location	On site	Simplest; base return is an option
Shift start	All at hour 0 (MVP); staggered as a lever	Keep the MVP simple, then show the gain
Idle teams may reposition	Yes, as a recommendation	README step T5
Duplicate incidents on one substation	Separate incidents	Keeps data as given
Travel model	Straight line × 1.3 at 50 km/h	Trips (~15-30 min) are much shorter than repairs (2-8 h)
Interface technology	Streamlit	Speed of delivery; suggested by the README

11. Limits and risks

- Synthetic customer counts and relocated incidents: results show the method's value, not real-world gains.
- Repair durations are taken as known; in reality they are uncertain (the bonus can address this with noise).
- No road blockages and no network topology: repairing one substation restores only that substation.
- All teams are interchangeable: the README mentions skills, but the data has none.
- Risk: spending too long on the interface or the LLM before the benchmark exists. Mitigation: phase gates in section 9.

12. Glossary FR / EN

Français	English	Français	English
poste	substation	dépannage	repair
coupure / incident	outage / incident	durée de dépannage (gamme)	standard repair duration
client sensible	sensitive customer (e.g. medical, critical site)	réalimentation	power restoration
équipe / agent	field team	prépositionnement	pre-positioning
site opérationnel	operational base	maintien en réserve	holding teams in reserve
rejeu	replay	saturation	no team available for waiting incidents